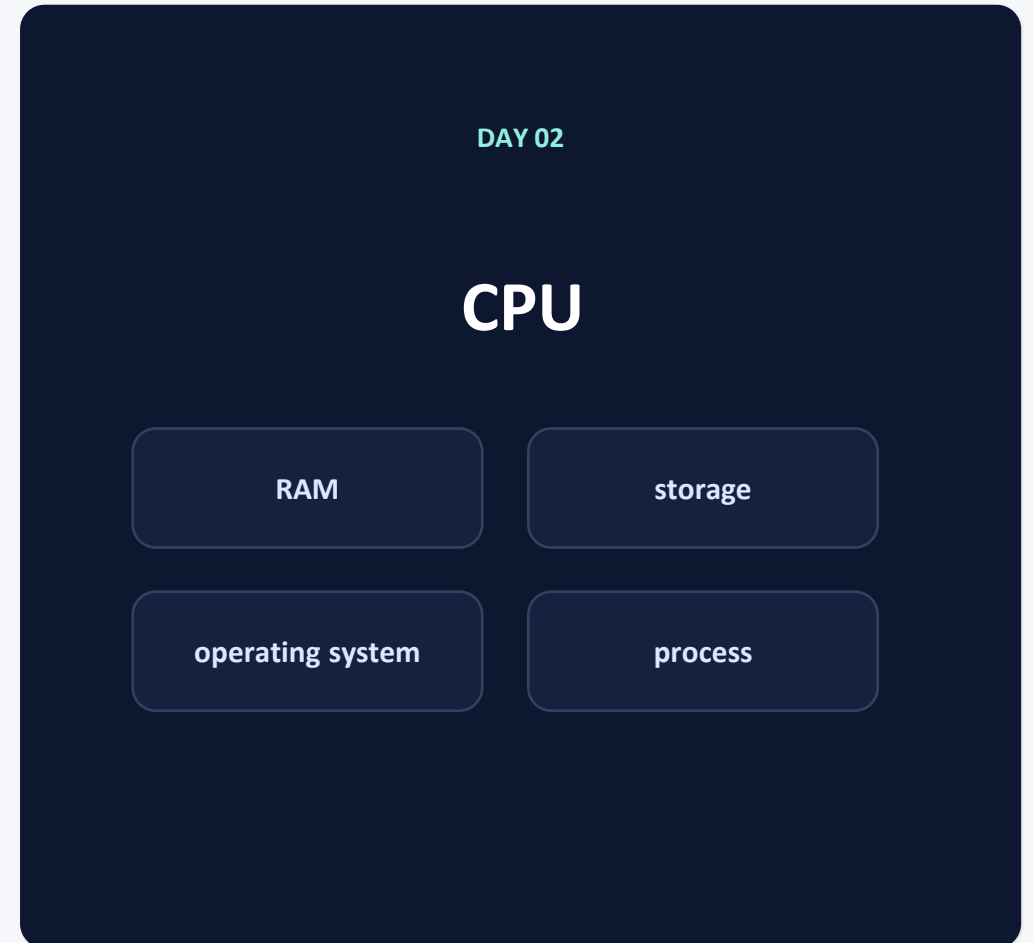


Как устроен компьютер

Объяснить, где физически живёт и выполняется программа: что делает CPU, зачем нужна RAM и чем она отличается от SSD.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Объяснить, где физически живёт и выполняется программа: что делает CPU, зачем нужна RAM и чем она отличается от SSD.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Карта мира IT и язык вакансий

Что уже есть в нашей модели.



СЕГОДНЯ

Как устроен компьютер

В День 1 мы увидели роли и стек. Теперь заглянем внутрь компьютера и поймём, где вообще выполняется код.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Когда понимаешь путь программы от файла на диске до процесса в памяти, ошибки вроде «не хватает памяти» или «программа зависла» перестают быть магией.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

CPU — сотрудник, RAM — рабочий стол, SSD — шкаф с документами. На стол кладём то, с чем работаем сейчас; в шкаф — то, что нужно сохранить надолго.

ПЕРЕВОД В IT

CPU

процессор, который выполняет инструкции программ

RAM

быстрая временная память для работающих программ

storage

накопитель для долговременного хранения файлов

ТЕОРИЯ

Слой 1: CPU • RAM • storage

Сначала смысл, затем профессиональный термин.

**CPU**

процессор, который выполняет инструкции программ

**RAM**

быстрая временная память для работающих программ

**storage**

накопитель для долговременного хранения файлов

Слой 2: operating system • process • file system

Сначала смысл, затем профессиональный термин.

operating system

система, управляющая оборудованием и приложениями

process

экземпляр программы, который сейчас выполняется

file system

правила организации файлов и папок на накопителе

Слой 3: core • cache

Сначала смысл, затем профессиональный термин.

**core**

отдельное вычислительное ядро процессора

**cache**

очень быстрая память рядом с процессором

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: SSD • driver

Сначала смысл, затем профессиональный термин.

**SSD**

быстрый накопитель без движущихся деталей

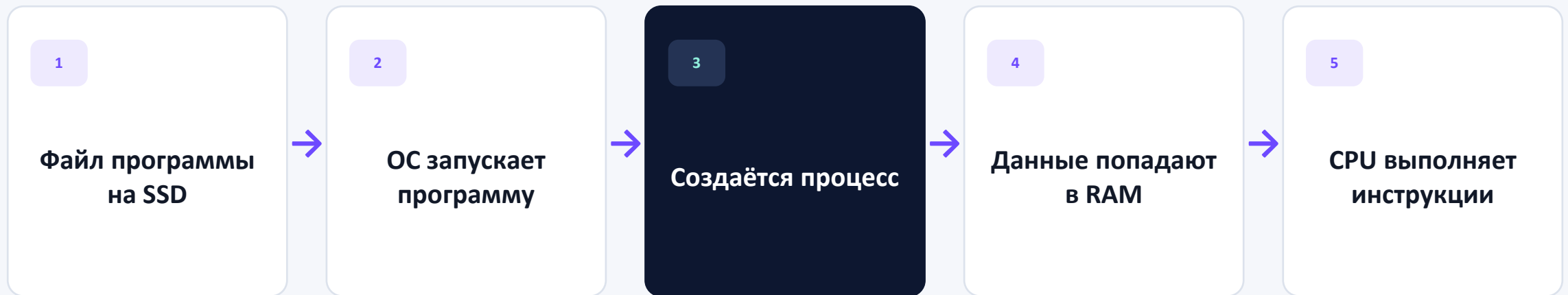
**driver**

программа для взаимодействия системы с устройством

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day02_step1

```
cpu = "выполняет команды"  
ram = "хранит временно"  
storage = "хранит долго"  
  
print(cpu)  
print(ram)  
print(storage)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
cpu = "выполняет команды"
```

Сохраняем значение или результат вычисления под понятным именем.

```
ram = "хранит временно"
```

Сохраняем значение или результат вычисления под понятным именем.

```
storage = "хранит долго"
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(cpu)
```

Показываем результат, чтобы увидеть состояние программы.

```
print(ram)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
cpu = "выполняет команды"  
ram = "хранит временно"  
storage = "хранит долго"  
  
print(cpu)  
print(ram)  
print(storage)
```

РЕЗУЛЬТАТ

выполняет команды
хранит временно
хранит долго

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
program = "calculator.py"  
place = "RAM"  
  
print(program, "сейчас работает из", place)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

calculator.py сейчас работает из RAM

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
ram = "временная память"  
print(RAM)
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Python различает регистр. Переменная называется `ram`, а `RAM` — другое, несуществующее имя. Поэтому возникнет `NameError`.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
ram = "временная память"  
print(ram)
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Показать путь файла `notes.py` от хранения до выполнения и не путать диск с оперативной памятью.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Файл `notes.py` лежит на SSD.**
- 2 ОС читает файл и запускает его.
- 3 Создаётся процесс.
- 4 Нужные данные процесса помещаются в RAM.
- 5 CPU выполняет инструкции процесса.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
file = "notes.py"  
storage = "SSD"  
process = "notes.exe"  
memory = "RAM"
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
file = "notes.py"
storage = "SSD"
process = "notes.exe"
memory = "RAM"

print(file, "лежит на", storage)
print(process, "работает, используя", memory)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

notes.py лежит на SSD
notes.exe работает, используя RAM

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Есть три ситуации: открытый документ, сохранённый файл и выполняемая инструкция. Определи, что ближе к RAM, SSD и CPU.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Спроси: это хранится долго, нужно прямо сейчас или уже выполняется?

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
items = {  
    "открытый документ": "RAM",  
    "сохранённый файл": "SSD",  
    "выполняемая инструкция": "CPU",  
}  
  
for item, place in items.items():  
    print(item, "->", place)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

открытый документ -> RAM
сохранённый файл -> SSD
выполняемая инструкция -> CPU

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Собери мини-схему запуска hello.py: от файла до выполнения процессором.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
steps = ["hello.py на SSD", "ОС создаёт процесс", "данные в RAM", "CPU выполняет"]  
  
for step in steps:  
    print(step)
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

hello.py на SSD
ОС создаёт процесс
данные в RAM
CPU выполняет

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Путать RAM и постоянное хранилище.

02 Считать, что CPU «хранит» файлы.

03 Думать, что программа и процесс — одно и то же.

04 Не учитывать роль операционной системы.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

- 1 **Что такое «CPU» и какую проблему это понятие помогает описать?**
- 2 **Чем «CPU» отличается от «operating system»? Объясни без копирования определения.**

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 CPU — процессор, который выполняет инструкции программ.

2 CPU: процессор, который выполняет инструкции программ. operating system: система, управляющая оборудованием и приложениями.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

**выполняет команды
хранит временно
хранит долго**

4

Python различает регистр. Переменная называется ram, а RAM — другое, несуществующее имя. Поэтому возникнет NameError.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Объяснить, где физически живёт и выполняется программа: что делает CPU, зачем нужна RAM и чем она отличается от SSD.**
- ✓ Читать и объяснять пример темы «Как устроен компьютер».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Интернет, клиент и сервер

Завтра: Интернет, клиент и сервер. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.